

PolyFlop8 Data RAM

Unit Test Card

Comprehensive Verification of Data Memory Operations

Document ID:	TC-RAM-001
Revision:	1.0
Date:	January 5, 2026
Test Level:	Unit/Block Level
Test Engineer:	Artem Horiunov
Status:	DRAFT

1 TEST OVERVIEW

1.1 Unit Under Test (UUT)

Component	Data RAM (Random Access Memory)
VHDL Entity	DataRAM
Source File	Processor.txt (DataRAM entity)
Test Environment	Vivado ML 2025
Memory Size	256 bytes (8-bit address, 0x00 to 0xFF)
Data Width	8-bit
Access Type	Asynchronous Read, Synchronous Write

1.2 Scope

This test card defines the verification procedure for the PolyFlop8 Data RAM component. The RAM provides storage for data during program execution, supporting multiple addressing modes and data sources as implemented in the VHDL code.

1.3 Key Architectural Features (Based on VHDL)

- **Asynchronous Read:** Data available immediately without clock edge
- **Synchronous Write:** Updates occur only on rising clock edge when `mem_we='1'`
- **Address MUX:** 4 address sources via 2-bit selector
- **Data MUX:** 3 data sources via 2-bit selector
- **Memory Array:** 256x8 bit array initialized to zeros

1.4 Address MUX Selection (`mem_addr_sel`)

Selector	Address Source	Signal Name	Typical Use
"00"	ALU Result	alu_out	Calculated address (Base + Offset)
"01"	Register R7 (X-Pointer)	data_reg7x	Indirect addressing via R7
"10"	Register A Value	regfile_data_a	Indirect addressing [Ra]
"11"	Immediate Value	rs_imm_data	Direct addressing [Imm]

1.5 Data MUX Selection (`mem_data_sel`)

Selector	Data Source	Signal Name	Typical Use
"00"	Register B Value	regfile_data_b	STORE instruction (ST)
"01"	Program Counter	pc_data	CALL instruction (return address)
"10"	ALU Result	alu_out	Direct ALU result storage

1.6 Test Objectives

- Verify asynchronous read behavior (no clock dependency)
- Validate synchronous write on rising clock edge when enabled
- Test all 4 address multiplexer selections
- Test all 3 data multiplexer selections
- Verify boundary behavior at addresses 0x00 and 0xFF
- Test memory retention and proper isolation between addresses
- Verify correct signal mapping from VHDL implementation

2 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-RAM-01	Verify asynchronous read returns value at selected address without clock	TC-RAM-001-01	TBD
TR-RAM-02	Verify synchronous write updates memory only on rising edge when mem_we='1'	TC-RAM-001-02	TBD
TR-RAM-03	Verify Address MUX selects correct source (ALU, R7, Reg A, Immediate)	TC-RAM-001-03	TBD
TR-RAM-04	Verify Data MUX selects correct write source (Reg B, PC, ALU)	TC-RAM-001-04	TBD
TR-RAM-05	Verify memory boundary behavior at 0x00 and 0xFF addresses	TC-RAM-001-05	TBD

Table 1: Requirements Coverage Matrix (from PolyFlop8 Test Specification)

3 DETAILED TEST CASES

3.1 Test Case TC-RAM-001-01: Asynchronous Read

Requirement: TR-RAM-01

Purpose: Verify asynchronous read returns the value stored at selected address without requiring a clock edge, as implemented in VHDL.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
1.1	Basic asynchronous read	1. Write 0xAA to address 0x10 (mem_we='1', clock ↑) 2. Set address to 0x10 (mem_we='0')	ram_data_out = 0xAA Immediately after address change
1.2	Address change during read	1. Write 0x55 to 0x20 2. Write 0x66 to 0x21 3. Address = 0x20 → 0x21	ram_data_out changes: 0x55 → 0x66 immediately

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
1.3	Read during write cycle	1. Address = 0x30 2. mem_we='1', data=0x77 3. clock ↑ (write occurs) 4. Read immediately	Before write: old value After write: 0x77 immediately
1.4	Continuous read during address scan	Pre-fill memory pattern Scan addresses 0x00-0x0F	Each address shows correct value without clock needed

3.2 Test Case TC-RAM-001-02: Synchronous Write

Requirement: TR-RAM-02

Purpose: Verify synchronous write updates memory only on rising clock edge when mem_we = '1', as per VHDL process.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
2.1	Write with all conditions met	mem_addr_sel = "00" alu_out = 0x40 mem_data_sel = "00" regfile_data_b = 0x99 mem_we = '1' clock ↑	Memory[0x40] = 0x99 Updated exactly on clock edge
2.2	Write with mem_we='0' (no write)	Same as 2.1 but mem_we = '0' clock ↑	No write occurs Memory[0x40] unchanged
2.3	Write on falling edge (no write)	Setup as 2.1 clock ↓	No write occurs Only rising edge triggers
2.4	Write with timing constraints	Change address/data just before clock edge Setup/hold time test	Write uses values at clock edge, not before/after
2.5	Back-to-back writes	Write 0x11 to 0x50 Write 0x22 to 0x51 Consecutive clock cycles	Each write succeeds on respective clock edge

3.3 Test Case TC-RAM-001-03: Address MUX Selection

Requirement: TR-RAM-03

Purpose: Verify Address MUX selects correct source based on mem_addr_sel (2 bits).

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
3.1	Select ALU address ("00")	mem_addr_sel = "00" alu_out = 0x50 data_reg7x = 0x60 regfile_data_a = 0x70 rs_imm_data = 0x80	Selected address = 0x50 (ALU source selected)
3.2	Select R7 address ("01")	mem_addr_sel = "01" alu_out = 0x51 data_reg7x = 0x61 regfile_data_a = 0x71 rs_imm_data = 0x81	Selected address = 0x61 (R7 pointer selected)
3.3	Select Reg A address ("10")	mem_addr_sel = "10" alu_out = 0x52 data_reg7x = 0x62 regfile_data_a = 0x72 rs_imm_data = 0x82	Selected address = 0x72 (Register A selected)

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
3.4	Select Immediate address ("11")	mem_addr_sel = "11" alu_out = 0x53 data_reg7x = 0x63 regfile_data_a = 0x73 rs_imm_data = 0x83	Selected address = 0x83 (Immediate selected)
3.5	Dynamic switching test	Cycle through all 4 modes while reading same location	Reads from different memory locations correctly

3.4 Test Case TC-RAM-001-04: Data MUX Selection

Requirement: TR-RAM-04

Purpose: Verify Data MUX selects correct write data source (2-bit selector).

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
4.1	Select Reg B data ("00") - STORE	mem_data_sel = "00" regfile_data_b = 0xA1 pc_data = 0xB1 alu_out = 0xC1 mem_we = '1' clock ↑	Written data = 0xA1 (Register B selected)
4.2	Select PC data ("01") - CALL	mem_data_sel = "01" regfile_data_b = 0xA2 pc_data = 0xB2 alu_out = 0xC2 mem_we = '1' clock ↑	Written data = 0xB2 (PC selected for return address)
4.3	Select ALU data ("10")	mem_data_sel = "10" regfile_data_b = 0xA3 pc_data = 0xB3 alu_out = 0xC3 mem_we = '1' clock ↑	Written data = 0xC3 (ALU result selected)
4.4	Invalid selector ("11")	mem_data_sel = "11" (undefined in VHDL)	Written data = 0x00 (default case in VHDL)

3.5 Test Case TC-RAM-001-05: Memory Boundary Behavior

Requirement: TR-RAM-05

Purpose: Verify memory boundary behavior for lowest (0x00) and highest (0xFF) addresses.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
5.1	Write/Read at address 0x00	1. Write 0x11 to 0x00 2. Read from 0x00	Memory[0x00] = 0x11 Read returns 0x11
5.2	Write/Read at address 0xFF	1. Write 0x22 to 0xFF 2. Read from 0xFF	Memory[0xFF] = 0x22 Read returns 0x22
5.3	Address wrap-around test	1. Set address to 0xFF 2. Read, then increment address externally	No automatic wrap: Address 0x100 is out of range (8-bit max = 0xFF)
5.4	Boundary isolation test	Write to 0x00 and 0xFF Verify middle addresses (0x01-0xFE) unchanged	Writes don't affect other addresses
5.5	Full memory pattern test	Write unique pattern to all 256 addresses	All addresses retain their unique values

4 TEST BENCH ARCHITECTURE

4.1 VHDL Testbench Structure

```
-- DataRAM Testbench (dataram_tb.vhd)
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity dataram_tb is
end dataram_tb;

architecture Behavioral of dataram_tb is
    -- Component Declaration (matches exact VHDL from Processor.txt)
    component DataRAM
        Port (
            clk                : in  STD_LOGIC;
            mem_we             : in  STD_LOGIC;
            mem_addr_sel       : in  STD_LOGIC_VECTOR(1 downto 0);
            mem_data_sel       : in  STD_LOGIC_VECTOR(1 downto 0);
            regfile_data_b     : in  STD_LOGIC_VECTOR (7 downto 0);
            alu_out             : in  STD_LOGIC_VECTOR (7 downto 0);
            pc_data            : in  STD_LOGIC_VECTOR (7 downto 0);
            data_reg7x         : in  STD_LOGIC_VECTOR (7 downto 0);
            regfile_data_a     : in  STD_LOGIC_VECTOR (7 downto 0);
            rs_imm_data        : in  STD_LOGIC_VECTOR (7 downto 0);
            ram_data_out       : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component;

    -- Test signals (match entity exactly)
    signal clk_tb                : STD_LOGIC := '0';
    signal mem_we_tb            : STD_LOGIC := '0';
    signal mem_addr_sel_tb      : STD_LOGIC_VECTOR(1 downto 0) := "00";
    signal mem_data_sel_tb      : STD_LOGIC_VECTOR(1 downto 0) := "00";
    signal regfile_data_b_tb    : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal alu_out_tb           : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal pc_data_tb           : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal data_reg7x_tb        : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal regfile_data_a_tb    : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal rs_imm_data_tb       : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal ram_data_out_tb      : STD_LOGIC_VECTOR(7 downto 0);

    constant CLK_PERIOD : time := 10 ns;

begin
    -- Unit Under Test Instantiation
    UUT: DataRAM port map (
        clk                => clk_tb,
        mem_we             => mem_we_tb,
        mem_addr_sel       => mem_addr_sel_tb,
        mem_data_sel       => mem_data_sel_tb,
        regfile_data_b     => regfile_data_b_tb,
        alu_out            => alu_out_tb,
        pc_data            => pc_data_tb,
        data_reg7x         => data_reg7x_tb,
        regfile_data_a     => regfile_data_a_tb,
        rs_imm_data        => rs_imm_data_tb,
        ram_data_out       => ram_data_out_tb
    );

    -- Clock generation process
    clk_process: process
```

```

begin
    clk_tb <= '0';
    wait for CLK_PERIOD/2;
    clk_tb <= '1';
    wait for CLK_PERIOD/2;
end process;

-- Main test stimulus process
stim_proc: process
begin
    wait for 100 ns;
    report "Starting Data RAM Test Bench";

    -- Test Case 1.1: Asynchronous Read
    report "Test Case 1.1: Asynchronous Read";
    -- First, write a value
    mem_addr_sel_tb <= "00";      -- Select ALU as address source
    alu_out_tb <= x"10";          -- Address 0x10
    mem_data_sel_tb <= "00";      -- Select Reg B as data source
    regfile_data_b_tb <= x"AA";   -- Data to write
    mem_we_tb <= '1';            -- Enable write
    wait until rising_edge(clk_tb);
    mem_we_tb <= '0';            -- Disable write

    -- Now read asynchronously
    wait for 5 ns;
    assert ram_data_out_tb = x"AA"
        report "Async read failed: Expected 0xAA, got " &
            to_hstring(ram_data_out_tb) severity error;

    -- Test Case 2.1: Synchronous Write
    report "Test Case 2.1: Synchronous Write";
    mem_addr_sel_tb <= "00";
    alu_out_tb <= x"20";
    mem_data_sel_tb <= "00";
    regfile_data_b_tb <= x"55";
    mem_we_tb <= '1';
    wait until rising_edge(clk_tb);
    wait for 1 ns; -- Small delay after clock edge

    -- Verify by reading
    mem_we_tb <= '0';
    wait for 5 ns;
    assert ram_data_out_tb = x"55"
        report "Write failed: Expected 0x55, got " &
            to_hstring(ram_data_out_tb) severity error;

    -- Continue with all other test cases...

    -- Test Case 3.1: Address MUX - ALU
    report "Test Case 3.1: Address MUX - ALU";
    mem_addr_sel_tb <= "00";
    alu_out_tb <= x"30";
    data_reg7x_tb <= x"31";
    regfile_data_a_tb <= x"32";
    rs_imm_data_tb <= x"33";
    -- Write to verify address selection
    mem_data_sel_tb <= "00";
    regfile_data_b_tb <= x"99";
    mem_we_tb <= '1';
    wait until rising_edge(clk_tb);
    mem_we_tb <= '0';

```

```
-- Read back - should be from address 0x30
wait for 5 ns;
assert ram_data_out_tb = x"99"
    report "Address MUX 00 failed" severity error;

-- Test Case 5.1: Boundary address 0x00
report "Test Case 5.1: Boundary address 0x00";
mem_addr_sel_tb <= "11"; -- Immediate address
rs_imm_data_tb <= x"00"; -- Address 0x00
mem_data_sel_tb <= "00";
regfile_data_b_tb <= x"11";
mem_we_tb <= '1';
wait until rising_edge(clk_tb);
mem_we_tb <= '0';

-- Read back
wait for 5 ns;
assert ram_data_out_tb = x"11"
    report "Boundary 0x00 failed" severity error;

report "All Data RAM test cases completed successfully!";
wait;
end process;

end Behavioral;
```

4.2 Test Sequence

1. **Initialization:** Reset all inputs, verify memory initialized to zeros
2. **Basic Read/Write:** Test fundamental memory operations
3. **Asynchronous Read Tests:** Verify immediate data availability
4. **Synchronous Write Tests:** Verify clock-edge-triggered writes
5. **Address MUX Tests:** Cycle through all 4 address sources
6. **Data MUX Tests:** Cycle through all 3 data sources
7. **Boundary Tests:** Test edge cases (0x00, 0xFF)
8. **Integration Tests:** Simulate realistic instruction sequences
9. **Corner Cases:** Test simultaneous read/write, timing edges

4.3 Monitoring and Logging

- Continuous monitoring of ram_data_out for every input change
- Log all MUX selections and corresponding outputs
- Track memory contents at key addresses
- Capture waveforms for timing verification
- Generate comprehensive test report

5 PASS/FAIL CRITERIA

5.1 Success Criteria

Criteria	Description
Functional	All test cases execute without assertion failures
Timing	Asynchronous reads immediate; writes only on rising clock edge
Coverage	100% of MUX paths tested (4 address × 3 data = 12 combinations)
Boundary	Correct behavior at 0x00 and 0xFF addresses
Data Integrity	Writes don't affect other memory locations
Interface	All ports match VHDL entity specification exactly

5.2 Failure Conditions

- Incorrect data returned for given address
- Write occurs without clock edge or with mem_we='0'
- Wrong MUX selection (address or data)
- Memory corruption or cross-talk between addresses
- Timing violations in read/write operations
- Any mismatch with VHDL implementation behavior

5.3 Coverage Goals

- **Statement Coverage:** 100% of DataRAM architecture
- **Branch Coverage:** All MUX selection paths
- **Toggle Coverage:** All address and data bits toggled 0→1 and 1→0
- **Memory Coverage:** All 256 addresses tested for read/write
- **Boundary Coverage:** Edge cases at 0x00 and 0xFF

6 SPECIAL TEST CONSIDERATIONS

6.1 Asynchronous vs Synchronous Behavior

- Verify read output changes immediately with address change (no clock)
- Confirm writes require rising clock edge AND mem_we='1'
- Test setup/hold times for write operations
- Monitor for glitches during address/data transitions

6.2 Memory Integrity Tests

- Verify writes to one address don't affect others
- Test memory retention across multiple clock cycles
- Check power-on/reset state (should be all zeros)
- Verify no address decoding errors

6.3 MUX Selection Verification

- Test all combinations of address and data MUX selections
- Verify MUX selections are independent of each other
- Test invalid selector values (should have defined behavior)
- Verify no cross-coupling between MUX inputs

6.4 Integration with Other Components

- Verify signal compatibility with Control Unit outputs
- Test interface with Register File (data_reg7x, regfile_data_a/b)
- Validate connection with ALU (alu_out as address/data source)
- Check PC interface for CALL instruction support

6.5 Performance Considerations

- Measure maximum read access time (should be combinational delay only)
- Verify write meets clock frequency requirements
- Test memory bandwidth with back-to-back operations
- Monitor for metastability in control signals

7 SIGN-OFF

7.1 Test Results Summary

Category	Metric	Target	Achieved
Requirements	Requirements Covered	5/5	
Test Cases	Test Cases Executed	18/18	
Coverage	Statement Coverage	100%	
Coverage	MUX Path Coverage	100%	
Coverage	Memory Address Coverage	100%	
Quality	Assertion Pass Rate	100%	
Timing	Read Access Time	<5ns	

7.2 Approval Signatures

Role	Name	Signature	Date
Test Engineer			
Lead Engineer			
Quality Assurance			
Project Manager			

7.3 Test Closure

- All 18 test cases executed and verified
- 100% statement, branch, and toggle coverage achieved
- Correct asynchronous read and synchronous write behavior confirmed
- All MUX selections function according to VHDL implementation
- Boundary addresses (0x00 and 0xFF) tested successfully
- All test artifacts archived per configuration management procedures
- Data RAM ready for integration testing